

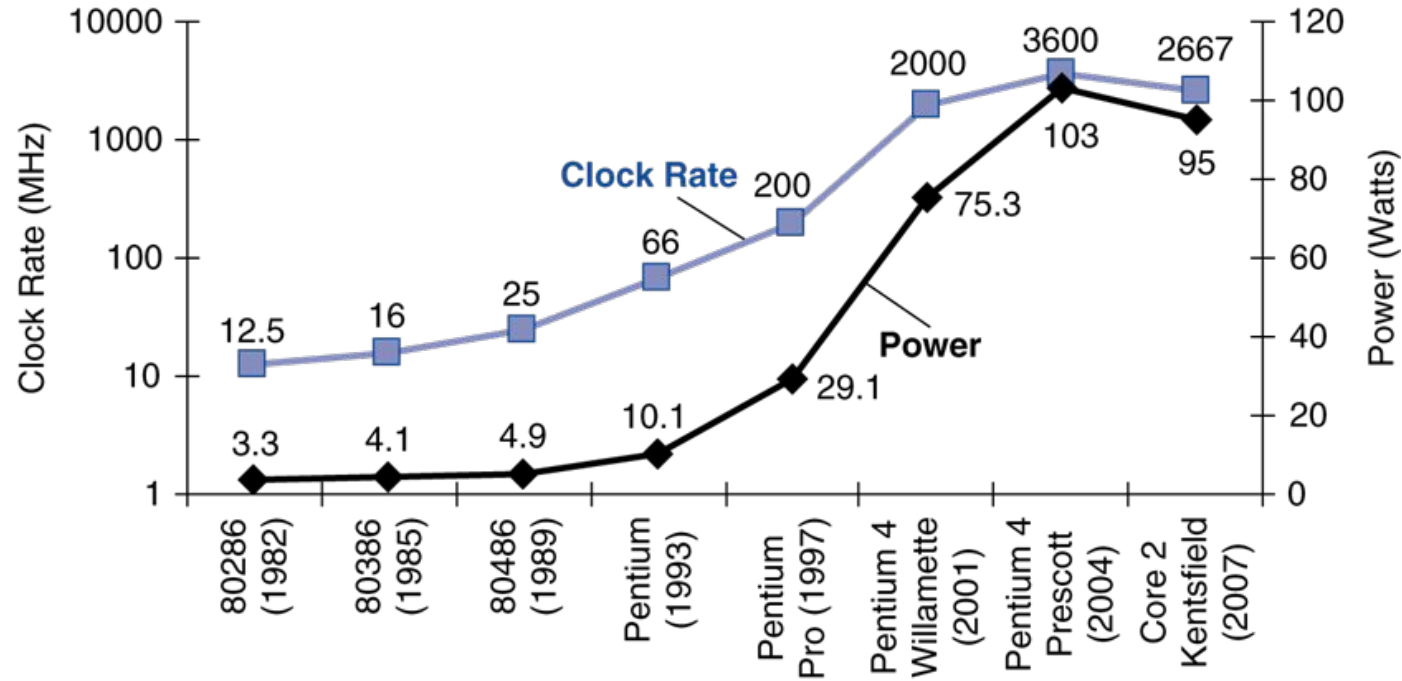


CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi

Lecturer, Department of Computer Science and Engineering
United International University

Power Trends



Formula for Power Trend



Power=Capacitive load×Voltage²×Frequency

Formula for Power Trend (Cont.)



Power = **Capacitive load** × Voltage² × Frequency

- Capacitive Load (C) – This represents the capacitance of the circuit or system. In electronic circuits, capacitors store and release charge, affecting power consumption.
- This depends on the design of the circuit (such as transistor size and interconnects). It can be optimized but is generally considered constant for a given architecture.

Formula for Power Trend (Cont.)



Power=Capacitive load×**Voltage²**×Frequency

This is the operating voltage of the system. The power consumption depends on the square of the voltage, meaning even a small increase in voltage can significantly increase power usage.

Threshold voltage is from 5 Voltage to 1 Voltage.

Formula for Power Trend (Cont.)



Power=Capacitive load×Voltage²×**Frequency**

This represents the switching frequency or the rate at which the circuit operates (e.g., clock frequency in a digital circuit). Higher frequencies lead to more frequent charging and discharging of capacitors, increasing power consumption.

Key Insights



Power consumption increases quadratically with voltage.
Reducing voltage can significantly lower power consumption.

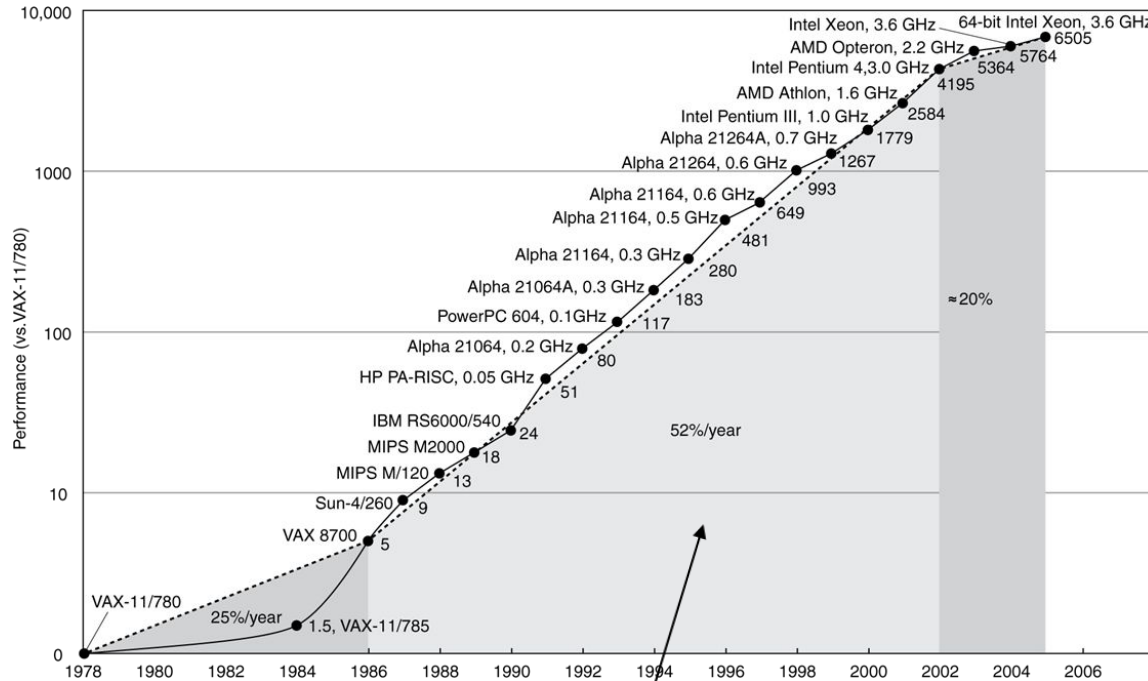
Power consumption increases linearly with capacitance and frequency, meaning higher capacitance or operating speeds result in more power dissipation.

Power Wall



The Power Wall is a fundamental limitation in computing where increasing a processor's performance by raising clock frequency (f) and voltage (V) leads to excessive power consumption and heat dissipation.

Problem with Uniprocessor



Constrained by power, instruction-level parallelism, memory latency

Solution using Multiprocessor



Multicore microprocessors

More than one processor per chip

Requires explicitly parallel programming

- Compare with instruction level parallelism
 - Hardware executes multiple instructions at once
 - Hidden from the programmer
 - Hard to do
 - Programming for performance
- Load balancing
- Optimizing communication and synchronization

Amdahl's Law

- T_{improved} → Total execution time after improvement.
- T_{affected} → The portion of execution time that is optimized (parallelized or improved).
- Improvement → The speedup factor applied to the affected portion (e.g., if a task is made 4× faster, improvement = 4).
- $T_{\text{unaffected}}$ → The portion of execution time that remains unchanged.

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

Amdahl's Law



- Improving an aspect of a computer and expecting a proportional improvement in overall performance

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- Example: multiply accounts for 80s/100s
 - How much improvement in multiply performance to get 5× overall?

$$20 = \frac{80}{n} + 20 \quad \text{■ Can't be done!}$$

- Corollary: make the common case fast



Thank you!